Συμβολοσειρές

«Ενσωματωμένο» Αντικείμενο

String

String
Αφηρημένος Τύπος Δεδομένων

- Αφηρημένη έννοια: Ακολουθία χαρακτήρων
  - Δεν μας ενδιαφέρει **πώς** υλοποιείται φτάνει να γνωρίζουμε **τι** μπορούμε να κάνουμε με strings

- Να θυμόμαστε ότι κάθε τύπος δεδομένων, συγκεκριμένος ή αφηρημένος, ορίζεται από:
  - Ένα σύνολο τιμών (που μπορεί να είναι «άπειρο»)
  - Ένα αριθμό λειτουργιών

Αφηρημένοι vs Συγκεκριμένοι Τύποι Δεδομένων

- Συγκεκριμένος Τύπος Δεδομένων
  - Γνωρίζουμε και το ‘πώς’ και το ‘τι’
  - Δεν υπάρχει απόκρυψη πληροφοριών
- Αφηρημένος Τύπος Δεδομένων
  - Γνωρίζουμε μόνο το ‘τι’
  - Υπάρχει απόκρυψη πληροφοριών
    - Αυτό παρέχει σημαντική ευελιξία, αφού το ‘πώς’ μπορεί να τροποποιείται χωρίς να επηρεάζει το ‘τι’
    - Η διεπαφή του τύπου δεδομένων προσδιορίζεται μέσω του ‘τι’

String

- Τεράστια χρησιμότητα
- Οποιαδήποτε ακολουθία χαρακτήρων ανήκει στο σύνολο τιμών του, ή όπως λέμε στην ορολογία των αντικειμένων, στο σύνολο των **περιπτώσεων** του (object **instances**)
- Κάθε κείμενο είναι μια περίπτωση String αφού δεν είναι τίποτα άλλο από μια ακολουθία χαρακτήρων
  - Το περιεχόμενο κάθε αρχείου είναι κείμενο (**αρχείο κειμένου** – text file)
  - Επομένως επιτρέπει την **επεξεργασία κειμένου**
  - Κείμενα αποτελούν βασική μορφή για **είσοδο** (input), π.χ. αρχεία κειμένου, και για **έξοδο** (output), π.χ. διακριτά σχήματα

String API – Application Programming Interface

public class String (Java string data type)

	Constructors	
	`String(String s)`	*create a string with the same value as* `s`
	Inspection	
`int`	`length()`	*string length*
`char`	`charAt(int i)`	*i th character*
`boolean`	`isEmpty()`	*is string empty?*
`boolean`	`isBlank()`	*is string whitespace only?*
	Content	
`String`	`substring(int i, int j)`	*i th through* `(j-1)` *st characters*
`boolean`	`contains(CharSequence s)`	*does string contain* `s`*?*
`boolean`	`startsWith(String pre)`	*does string start with* `pre`*?*
`boolean`	`endsWith(String post)`	*does string end with* `post`*?*
	Search	
`int`	`indexOf(String p)`	*index of first occurrence of* `p`
`int`	`indexOf(String p, int i)`	*index of first occurrence of* `p` *after* `i`
	Manipulation	
`String`	`concat(String t)`	*this string with* `t` *appended*
`String`	`repeat(int count)`	*string repeated* `count` *times*
`String`	`replaceAll(String a, String b)`	*replace all* `a`*s with* `b`*s (regex)*
`String[]`	`split(String delim)`	*split by occurrences of* `delim`
`String`	`strip()`	*string without leading/trailing whitespace*
	Output & Transformation	
`String`	`toLowerCase()`	*this string in lowercase*
`Stream<String>`	`lines()`	*stream of lines from this string*
`String`	`formatted(Object... args)`	*format this string (instance method)*
`<R> R`	`transform(Function f)`	*apply function* `f` *to this string*
	Comparison	
`boolean`	`equals(Object o)`	*is this string equal to* `o`*?*
`int`	`compareTo(String t)`	*string comparison*

Μέθοδοι αναφοράς – reference/instance methods

- ΑΤΔ χρησιμοποιούν μεθόδους αναφοράς–reference/instance methods-εφαρμόζονται σε συγκεκριμένο String

```
String s = "Hi!";
s.length();
```

•ΕΙΝΑΙ ΛΑΘΟΣ ΠΕΛΑΤΗΣ ΤΗΣ ΚΛΑΣΗΣ να καλέσει: String.length(); // το String δεν είναι object

- Μέθοδός με το όνομα της κλάσης είναι κατασκευαστής (δημιουργεί στιγμιότυπο, αντικείμενο)

```
String s = new String("foo"); // Δεσμεύει
```

χώρο στην μνήμη για ένα string και επιστρέφει την διεύθυνση του
•Αυτό που επιστρέφει ο κατασκευαστής το αναθέτουμε συνήθως σε μεταβλητή αναφοράς (δείχνει στο string)

Περιορισμός:
Δημιουργία αλλά όχι Τροποποίηση

- **Από τις παρεχόμενες λειτουργίες προκύπτει ότι:**
  - Αφού κατασκευάσουμε (μια περίπτωση) String δεν μπορούμε, δηλαδή δεν μας παρέχεται η δυνατότητα μέσω κάποιας λειτουργίας, να αλλάξουμε κάποιο στοιχείο του (να αλλάξουμε δηλαδή κάποιο χαρακτήρα του).
  - Μπορούμε να το αλλάξουμε, με το να το ξαναδημιουργήσουμε:

```
String s = "This is a string";
s = s.substring(0,1) + s.charAt(s.length()-1);
```

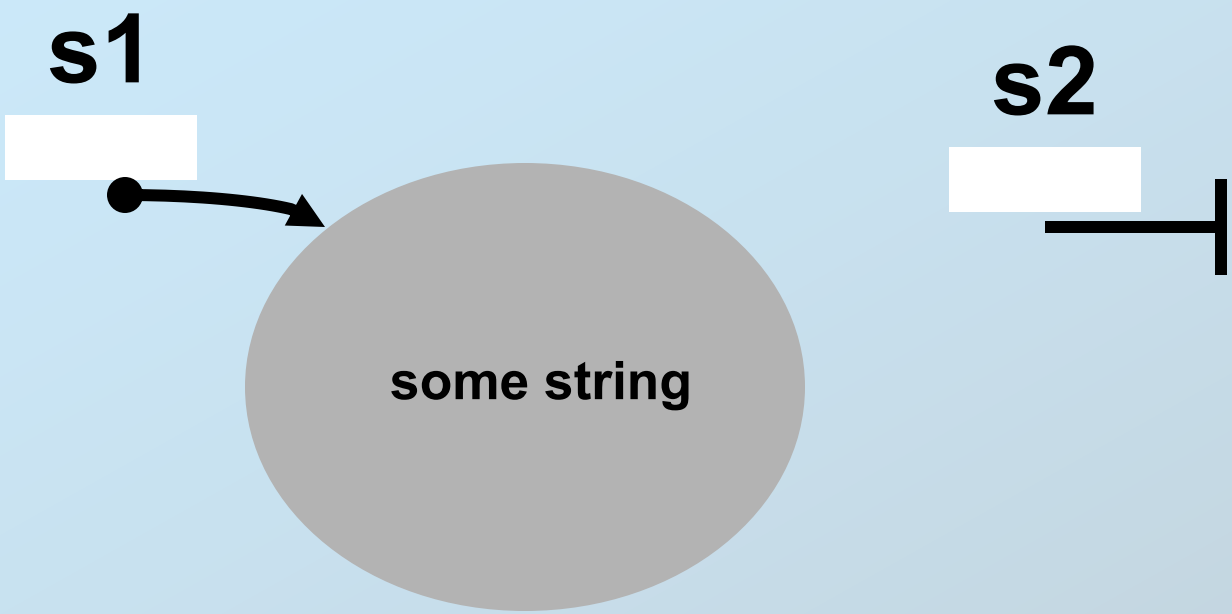
  - Και βεβαίως μπορούμε να δημιουργήσουμε/κατασκευάσουμε άλλα strings από αυτό:

```
String s1 = "Another string";
String s2 = s1.substring(0,8);
```

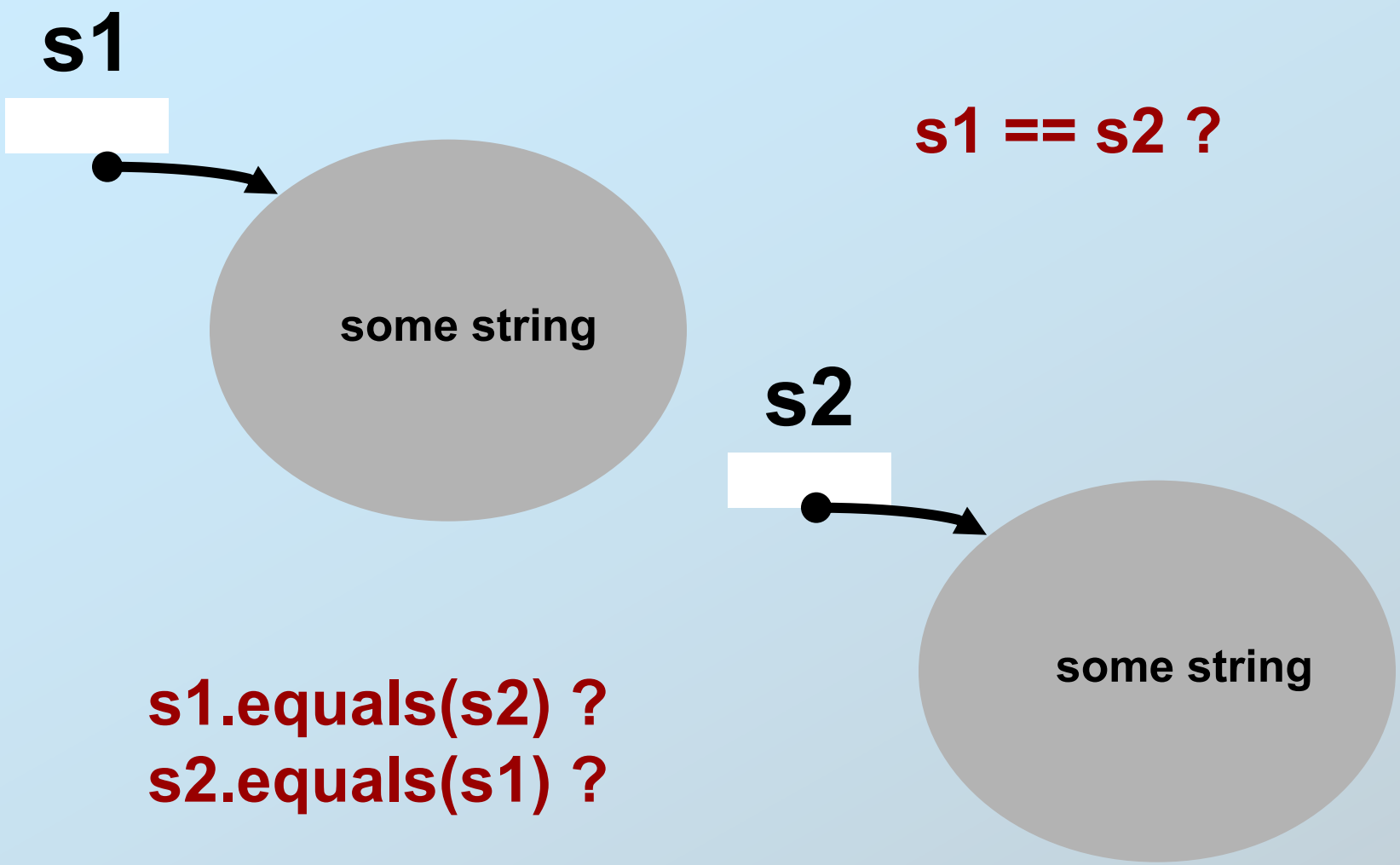
Επιπρόσθετα, όπως οι πίνακες, και οι περιπτώσεις αντικειμένων

είναι δείκτες

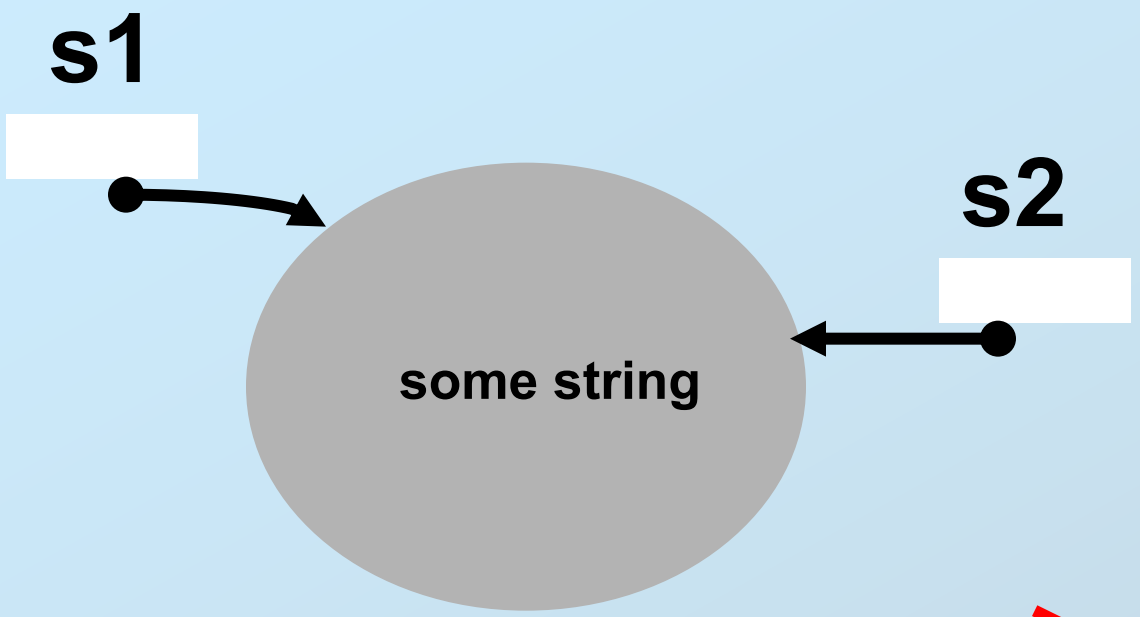
String s1 = “some string”;
String s2;



s2 = new String(s1);

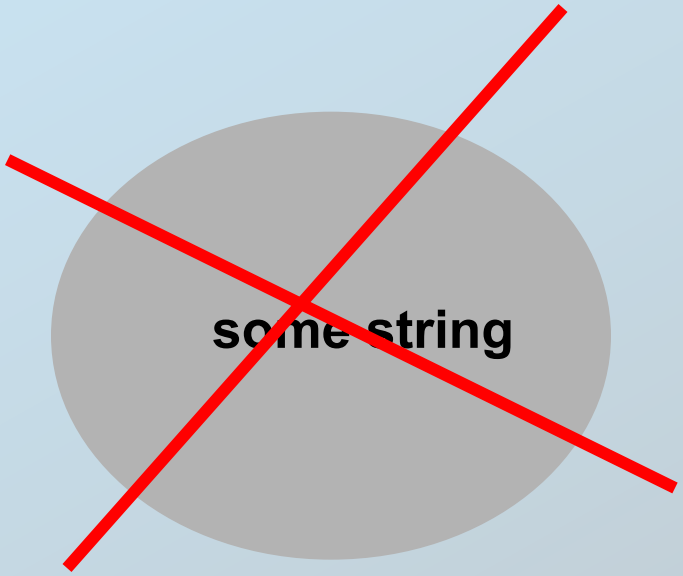


s2 = s1;



s1 == s2 ?

s1.equals(s2) ?
s2.equals(s1) ?



```
public class StringTest {

 public static void main(String[] args){
   String s1 = "some string";
   String s2; // null pointer
   s2 = new String(s1); // copy constructor

   if (s1==s2) System.out.println("Same instance");
   else System.out.println("Not same instance");
   if (s1.equals(s2)) System.out.println("Same value");
   else System.out.println("Not same value");

   s2 = s1;

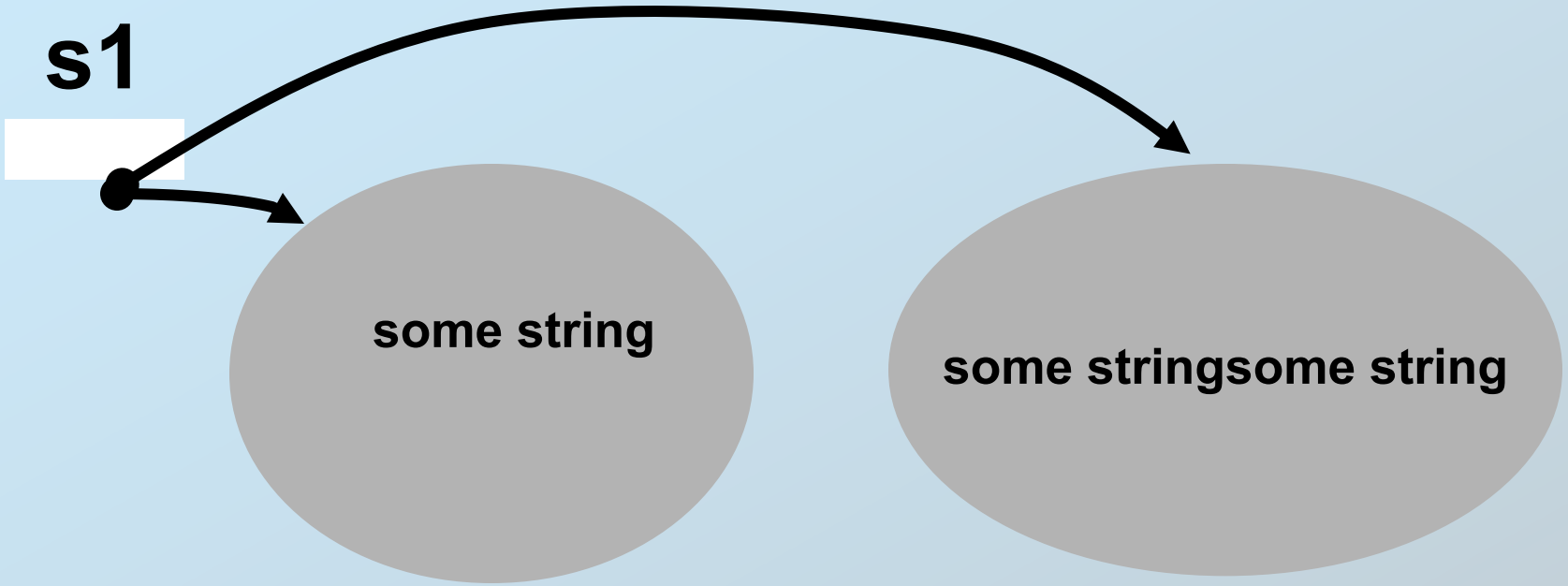
   if (s1==s2) System.out.println("Same instance");
   else System.out.println("Not same instance");
   if (s1.equals(s2)) System.out.println("Same value");
   else System.out.println("Not same value");
 }
}
```

```
$ java StringTest
Not same instance
Same value
Same instance
Same value
```

```
String s1 = “some string”; // initialized
String s2; // null pointer – not initialized

s2 += “some value”; // error – s2 not initialized
if (s2.equals(s1)) {..} // error – s2 not initialized

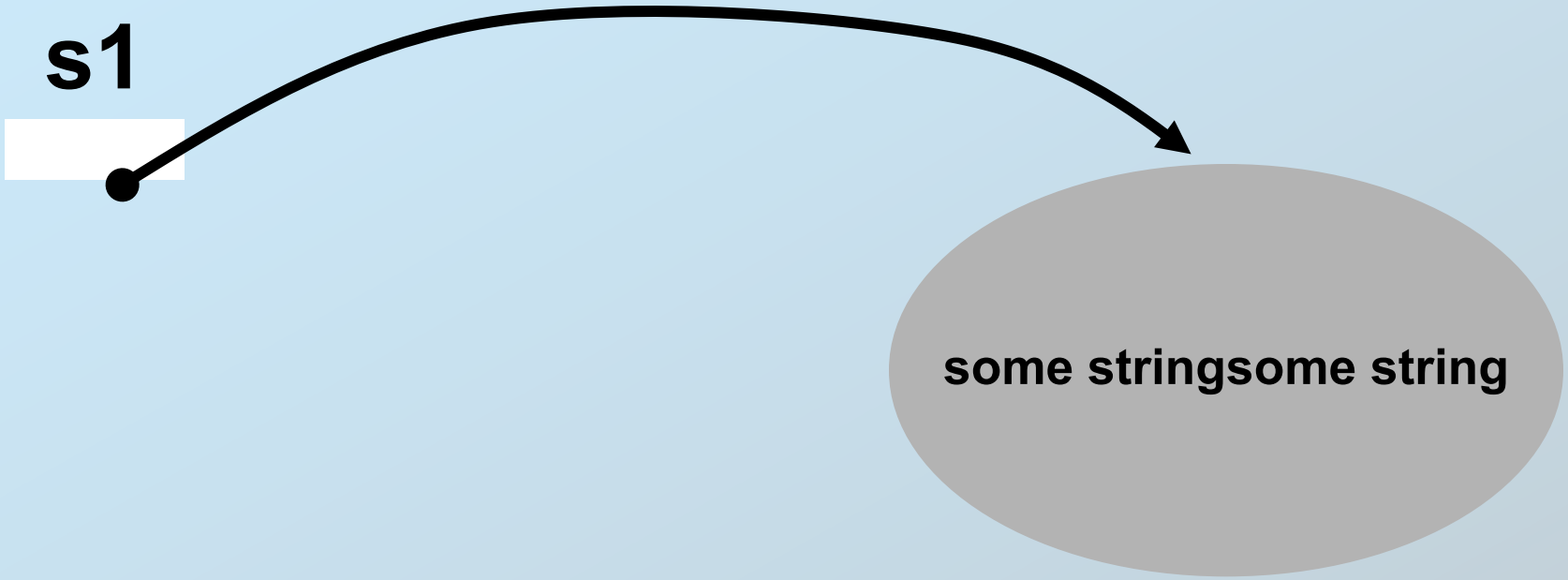

s1 += s1; // s1 is recreated!
```



```
String s1 = “some string”; // initialized
String s2; // null pointer – not initialized

s2 += “some value”; // error – s2 not initialized
if (s2.equals(s1)) {..} // error – s2 not initialized


s1 += s1; // s1 is recreated!
```



Παραδείγματα

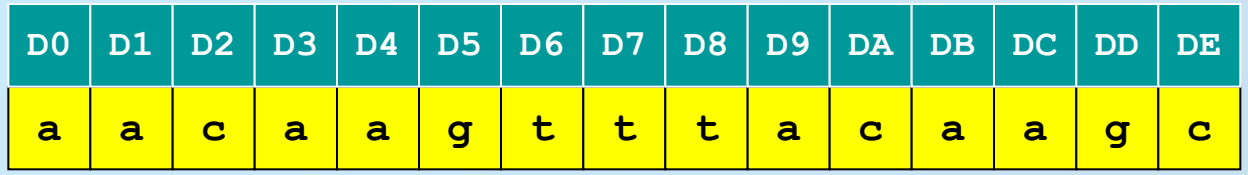
```
String s1 = "xy1";
String s2 = "xy1";
String s3 = "xy1a"
String s4 = "wa";

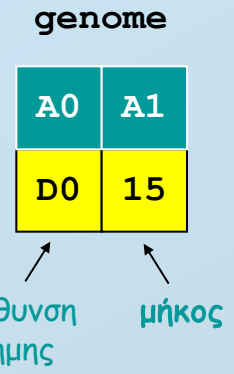
s1.compareTo(s2); // 0
s1.compareTo(s3); // <0
s1.compareTo(s4); // >0

//λεξικογραφική σύγκριση
```

```
String genome = "aacaagtttacaagc";
```

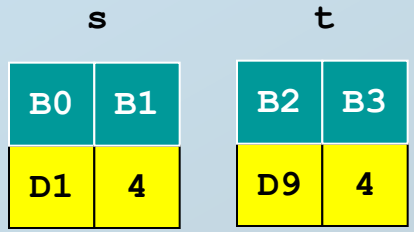
Πιθανή αναπαράσταση της μνήμης για μία συμβολοσειρά






```
String s = genome.substring(1, 5);
String t = genome.substring(9, 13);
```

s και t είναι ξεχωριστές συμβολοσειρές, οι οποίες απλά μοιράζονται την ίδια τιμή, “acaa”



`(s == t)` είναι **`false`**, αλλά `(s.equals(t))` είναι **`true`**.

Ο τελεστής == συγκρίνει δείκτες, ενώ η μέθοδος equals συγκρίνει ακολουθίες χαρακτήρων.

Συγκρίνει δείκτες!

Standard Είσοδος και Έξοδος

Standard είσοδος: Παρέχεται η βιβλιοθήκη `StdIn`

public class StdIn	
boolean isEmpty()	`true` *if no more values,* `false` *otherwise*
int readInt()	*read a value of type* `int`
double readDouble()	*read a value of type* `double`
long readLong()	*read a value of type long*
boolean readBoolean()	*read a value of type* `boolean`
char readChar()	*read a value of type* `char`
String readString()	*read a value of type* `String`
String readLine()	*read the rest of the line*
String readAll()	*read the rest of the text*

Standard έξοδος: Παρέχεται η βιβλιοθήκη `StdOut` (ταυτόσημη με τη βιβλιοθήκη `System.out`)

public class StdOut	
void print(String s)	*print* `s`
void println(String s)	*print* `s`*, followed by newline*
void println()	*print a new line*
void printf(String f, ...)	*formatted print*

Στατικές Μέθοδοι – public static

Παραδείγματα

is the string a palindrome?	`public static boolean isPalindrome(String s)`
	`{`
	`  int N = s.length();`
	`  for (int i = 0; i < N/2; i++)`
	`  if (s.charAt(i) != s.charAt(N-1-i))`
	`  return false;`
	`  return true;`
	`}`

extract file name and extension from a command-line argument	`String s = args[0];`
	`int dot = s.indexOf(".");`
	`String base = s.substring(0, dot);`
	`String extension = s.substring(dot + 1, s.length());`

print all lines in standard input that contain a string specified on the command line	`String query = args[0];`
	`while (!StdIn.isEmpty())`
	`{`
	`  String s = StdIn.readLine();`
	`  if (s.contains(query)) StdOut.println(s);`
	`}`

print all the hyperlinks (to educational institutions) in the text file on standard input

```
while (!StdIn.isEmpty())
{
   String s = StdIn.readString();
   if (s.startsWith("http://") && s.endsWith(".edu"))
      StdOut.println(s);
}
```

Επεξεργασία Αρχείου: Πλήθος γραμμών

```
public class CountLines {

 public static void main(String[] args){
   int count = 0;
   while (!StdIn.isEmpty()){
     StdIn.readLine(); count++;
   }

   System.out.println("Number of lines = " + count);
 }

}
```

```
$ java CountLines < CountLines.java
Number of lines = 12
```

Επεξεργασία Αρχείου: Πλήθος ‘λέξεων’

```
public class CountWords {

 public static void main(String[] args){
   int count = 0;
   while (!StdIn.isEmpty()){
     StdIn.readString(); count++;
   }

   System.out.println("Number of words = " + count);
 }

}
```

```
$ java CountWords < CountLines.java
Number of words = 27
```

Επεξεργασία Αρχείου:
Πλήθος γραμμών και πλήθος ‘λέξεων’

```
$java CountWordsLines < CountLines.java
Number of lines = 12
Number of words = 27
```

```
public class CountWordsLines {

 public static boolean white_space(char c){
   return c == ' ' || c == '\n' || c == '\t';
 }

 public static void main(String[] args){
   int lineCount = 0, wordCount = 0;
   while (!StdIn.isEmpty()){
     String s = StdIn.readLine(); lineCount++;
     boolean beg = false;
     for (int i = 0; i < s.length(); i++){
      if (beg && white_space(s.charAt(i)))
          { beg = false; wordCount++; }
      if (!beg && !white_space(s.charAt(i)))
          { beg = true; }
     }
     if (beg) wordCount++;
   }

   System.out.println("Number of lines = " + lineCount);
   System.out.println("Number of words = " + wordCount);
 }
}
```

Παράδειγμα: Εύρεση Γονιδίου

•Προ-γονιδιακή περίοδος. Ακολουθία ενός ανθρώπινου γονιδιώματος.

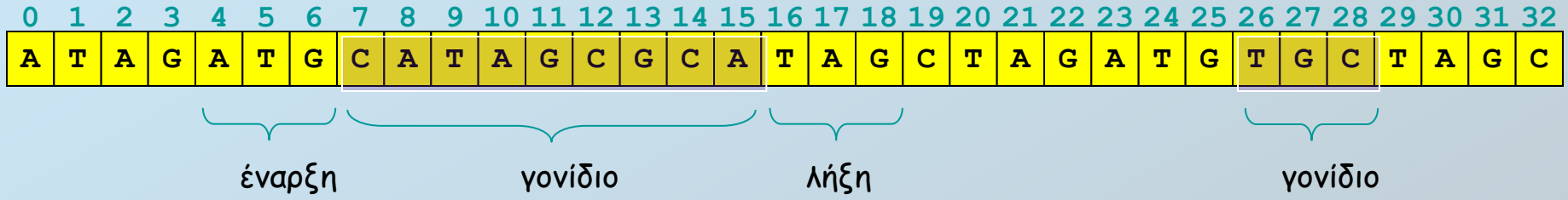
•Μετά-γονιδιακή περίοδος. Ανάλυση των δεδομένων και κατανόηση της δομής.

•Γονιδιωματική. Τα γονιδιώματα αντιπροσωπεύονται ως συμβολοσειρές σε σχέση με το αλφάβητο {A, C, T, G}

•Γονίδιο. Μία υποσυμβολοσειρά του γονιδιώματος το οποίο αντιπροσωπεύει μία λειτουργική μονάδα.

- Προηγείται από ATG. [κωδικόνιο έναρξης]
- Πολλαπλάσια των 3 νουκλεοτίδια. [κωδικόνιο εκτός έναρξης και τερματισμού]
- Ακολουθείται από TAG, TAA, ή TGA. [κωδικόνιο τερματισμού]

Στόχος: Να βρούμε όλα τα γονίδια



Εύρεση γονιδίου: Αλγόριθμος

•Αλγόριθμος: Σαρώστε από αριστερά προς τα δεξιά εντός του γονιδιώματος.

– Εάν εντοπισθεί το κωδικόνιο έναρξης, τότε θέστε `beg` στο δείκτη `i`.

– Εάν εντοπισθεί το κωδικόνιο τερματισμού και η υποσυμβολοσειρά είναι πολλαπλάσια του 3

- Γονίδιο εξόδου
- Επαναφορά του `beg` στο -1

start

stop

multiple of 3

i	codon	beg	*output*	*remaining portion of input string*
0		-1		ATAGATGCATAGCGCATAGCTAGATGTGCTAGC
1	TAG	-1		TAGATGCATAGCGCATAGCTAGATGTGCTAGC
4	ATG	4		ATGCATAGCGCATAGCTAGATGTGCTAGC
9	TAG	4		TAGCGCATAGCTAGATGTGCTAGC
16	TAG	4	CATAGCGCA	TAGCTAGATGTGCTAGC
20	TAG	-1		TAGATGTGCTAGC
23	ATG	23		ATGTGCTAGC
29	TAG	23	TGC	TAGC

Εύρεση γονιδίου: Υλοποίηση

```
public class GeneFind {
   public static void main(String[] args) {
      String start  = args[0];
      String stop   = args[1];
      String genome = StdIn.readAll();

      int beg = -1;
      for (int i = 0; i < genome.length() - 2; i++) {
         String codon = genome.substring(i, i+3);
         if (codon.equals(start)) beg = i;
         if (codon.equals(stop) && beg != -1) {
            String gene = genome.substring(beg+3, i);
            if (gene.length() % 3 == 0) {
               StdOut.println(gene);
               beg = -1;
            }
         }
      }
   }
}
```

```
% more genomeTiny.txt
ATAGATGCATAGCGCATAGCTAGATGTGCTAGC

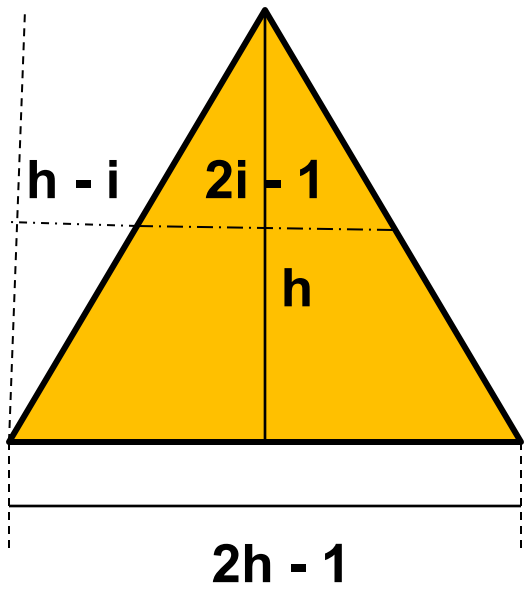
% java GeneFind ATG TAG < genomeTiny.txt
CATAGCGCA
TGC
```

Διακριτά Σχήματα: Πυραμίδες

```
$ java Pyramid 5 x
    x
   xxx
  xxxxx
 xxxxxxx
xxxxxxxxx

$ java Pyramid 7 o
      o
     ooo
    ooooo
   ooooooo
  ooooooooo
 ooooooooooo
ooooooooooooo

$ java Pyramid 9 p
        p
       ppp
      ppppp
     ppppppp
    ppppppppp
   ppppppppppp
  ppppppppppppp
 ppppppppppppppp
ppppppppppppppppp
```



```
public class Pyramid {

 public static String line (int len, char c){
    String s = "";
    for (int i = 1; i <= len; i++) s += c;
    return s;
 }

 public static String pyrRow (int h, int r, char c){
   return line(h-r, ' ') + line(2*r - 1, c) + line(h-r, ' ');
 }

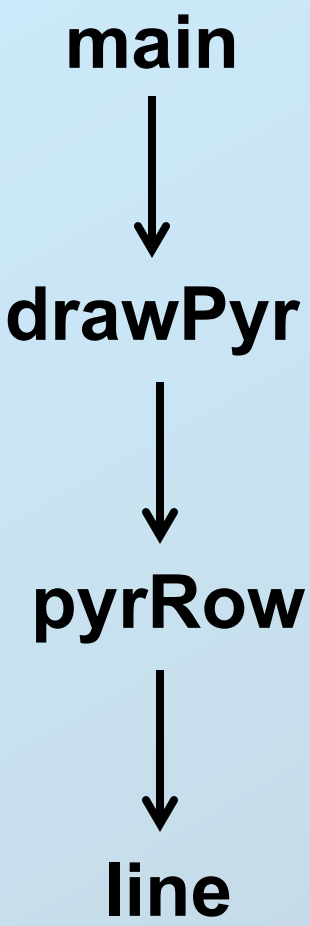
 public static String drawPyr (int h, char c){
   String s = "";
   for (int i = 1; i <= h; i++) s += pyrRow(h,i,c) + "\n";
   return s;
 }

 public static void main (String[] args){
   int h = Integer.parseInt(args[0]);
   char c = args[1].charAt(0);
   System.out.println(drawPyr(h,c));
 }
}
```

Ανάλυση Προβλήματος

Διάσπαση και σύνθεση

Διαίρει και Βασίλευε



Ιεραρχική Δομή

Επαναχρησιμοποίηση

Το ίδιο πρόγραμμα χωρίς άρθρωση

```
public class Pyramid {

 public static void main (String[] args){
   int h = Integer.parseInt(args[0]);
   char c = args[1].charAt(0);

   for (int r = 1; r <= h; r++){
     for (int j = 1; j <= h-r; j++) System.out.print(' ');
     for (int j = 1; j <= 2*r-1; j++) System.out.print(c);
     System.out.print('\n');
   }

 }

}
```

Το μη αρθρωτό πρόγραμμα με χρήση String

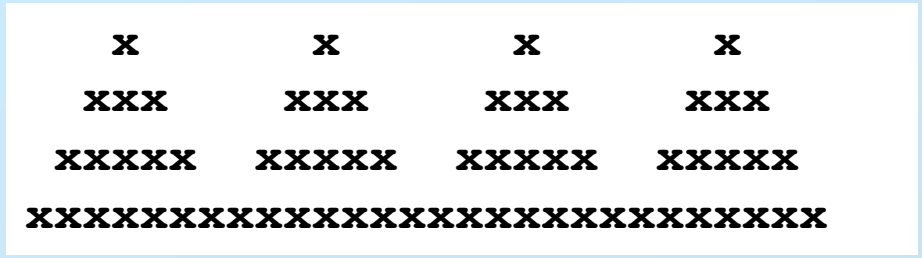
```
public class Pyramid {

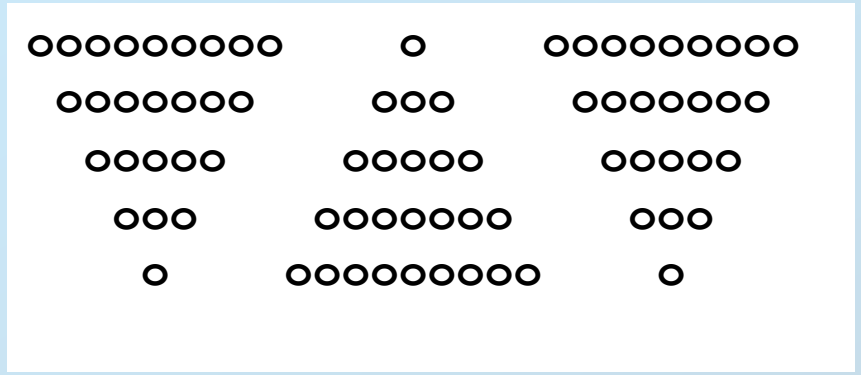
 public static void main (String[] args){
   int h = Integer.parseInt(args[0]);
   char c = args[1].charAt(0);
   String s = "";
   for (int r = 1; r <= h; r++){
     for (int j = 1; j <= h-r; j++) s += ' ';
     for (int j = 1; j <= 2*r-1; j++) s += c;
     s +='\n';
   }
  System.out.print(s);
 }

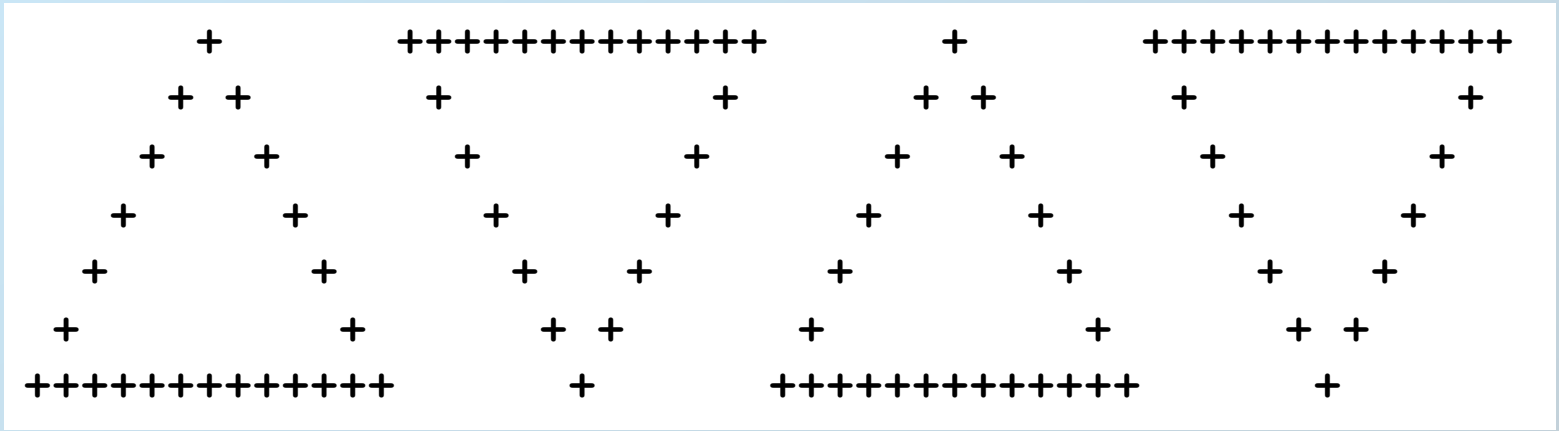
}
```

Ποιο από τα δύο προγράμματα (αρθρωτό ή μη αρθρωτό) θα μπορούσε πιο εύκολα να επεκταθεί/τροποποιηθεί για να μπορεί να κάνει τα ακόλουθα διακριτά σχήματα;

ή οποιοδήποτε συνδυασμό αυτών;







Παράδειγμα: Πίνακες με Στοιχεία Συμβολοσειρές

```
public class TabLib {

  public static String[] add(String[] ts, String s){
   if (ts == null) {
     ts = new String[1];
     ts[0] = s;
     return ts;}
   String[] nts = new String[ts.length+1];
   nts[0] = s;
   for (int i = 0; i < ts.length; i++){
      nts[i+1] = ts[i];
   }
   return nts;
  }

 public static String[] remove(String[] ts, String s){
    String[] nts = null;
    for (String s2 : ts){
     if (!s2.contains(s)) nts = add(nts, s2);
    }
    return nts;
  }
```

```
public static String[] select(String[] ts, String s){
   String[] nts = null;
   for (String s2 : ts){
    if (s2.contains(s)) nts = add(nts, s2);
   }
   return nts;
 }

 public static void display (String[] ts){
   for (String s : ts) System.out.println(s);
 }

 public static void reverse (String[] ts){
   for (int i=0, j=ts.length-1; i < j; i++, j--){
     String s = ts[i]; ts[i] = ts[j]; ts[j] = s;
   }
 }
```

```
public static void main(String[] args){
   String[] tab = null;
   while (!StdIn.isEmpty()){
      tab = add(tab, StdIn.readLine());
   }
   String[] nullLines = select(tab, "null");
   String[] returnLines = select(tab, "return");
   display(nullLines);
   System.out.println();
   display(returnLines);
 }
}
```

```
$java TabLib < TabLib.java
   if (ts == null) {
    String[] nts = null;
    String[] nts = null;
   String[] tab = null;
   String[] nullLines = select(tab, "null");
   display(nullLines);

     return ts;}
   return nts;
    return nts;
    return nts;
   String[] returnLines = select(tab, "return");
   display(returnLines);
```

Σύνοψη

- Το String ως «ενσωματωμένο» αντικείμενο
- Τεράστια χρησιμότητα
  - Επεξεργασία κειμένου
  - Είσοδος/Έξοδος
- Μέθοδοι αναφοράς – διεπαφή String
  - Περιπτώσεις String είναι δείκτες
  - Δημιουργία αλλά όχι τροποποίηση
- Παραδείγματα
  - Επεξεργασία αρχείων κειμένου
  - Εύρεση γονιδίου
  - Διακριτά σχήματα
  - Πίνακες με στοιχεία συμβολοσειρές

Στο βιβλίο

- **Ενότητα 1.2 – μερικώς**
- **Ενότητα 3.1**